

Proyecto: “DeezerEuropa Graph”

David Hernando Avila Jimenez ^{a,c}, Erica Marcela Martínez Silva ^{a,c}, Lina Marcela Rivas^{a,c},
Raúl Eduardo Vásquez Duarte ^{a,c}, Sebastián Amaya Porras^{a,c}

Sergio Alberto Mora Pardo^{b,c}

^a*Estudiantes de la Maestría en Analítica para Inteligencia de negocios*

^b*Profesor, Departamento de Ingeniería Industrial*

^c*Pontificia Universidad Javeriana, Bogotá, Colombia*

1. ENTENDIMIENTO DE NEGOCIO

La industria musical ha experimentado transformaciones desde sus inicios, especialmente en los métodos de almacenamiento y consumo de música. Desde la invención del fonógrafo y los tocadiscos, pasando por los casetes, CDs y USB, hasta la llegada de las plataformas digitales, las cuales revolucionaron la forma como escuchamos música y como expresamos las preferencias; es así como los servicios de Deezer, Spotify, Apple Music y YouTube han ganado protagonismo. Según datos suministrados por Statista¹, en 2023, Spotify se consolidaba como líder del mercado con 226 millones de suscriptores, mientras que Deezer ocupaba el octavo lugar con un total de 9.3 millones de usuarios suscritos. Por lo tanto, este cambio en la forma de consumo refleja una evolución tecnológica y una transformación a nivel de las preferencias del consumidor y en las estrategias de distribución musical.

Deezer es un servicio de transmisión de música de origen francés que comenzó a operar en 2007, desde entonces la plataforma ha expandido su biblioteca para ofrecer a los usuarios acceso a millones de pistas y funciones sociales, lo que les ha permitido a sus usuarios interactuar, compartir sus gustos y pistas favoritas. Su modelo de negocio se basa en suscripciones pagadas y anuncios en la versión gratuita, permitiendo a los usuarios elegir entre un acceso completo sin interrupciones o un servicio gratuito con anuncios. Además, la personalización de la experiencia del usuario es un aspecto importante, ya que estos esperan recomendaciones basadas en sus preferencias musicales y en las de sus amigos.

Por lo tanto, la mejora en su capacidad de personalización y recomendación representan un reto para Deezer, teniendo en cuenta el avance por parte de otras plataformas y las demandas actuales del mercado. Es así, como la predicción del género de los usuarios, representa un aspecto clave para generar recomendaciones musicales relevantes y al no contar con información explícita sobre cada usuario, Deezer debe inferir esta variable a partir de patrones de comportamiento, como las listas de reproducción del usuario y sus relaciones de seguimiento mutuo entre los usuarios.

Por tal razón, la empresa necesita encontrar nuevas formas de diferenciarse y atraer nuevos usuarios, así como técnicas para retener a los actuales. Para abordar estos retos, Deezer puede beneficiarse del uso de métodos avanzados Machine Learning como el uso de técnicas de Graph Neural Network. En este contexto, la predicción del género musical y de los usuarios a través de técnicas avanzadas de aprendizaje automático representa una oportunidad significativa para la industria de la música y las plataformas de streaming. La

¹ Fuente: <https://es.statista.com/estadisticas/942349/principales-plataformas-de-musica-en-streaming-del-mundo-segun-suscriptores/>

capacidad de entender las preferencias por género y la relación e interacción entre los usuarios permite a las empresas adaptar sus catálogos y promociones, potenciando así su rentabilidad y la satisfacción del cliente. Para el entendimiento a nivel interno y externo del sector se llevó el estudio de 2 metodologías diferentes PESTEL y DOFA.



Ilustración 1. Matriz DOFA. Elaboración propia

Ilustración 2. Análisis PESTEL. Elaboración propia



1.1 Contexto de negocio.

1.1.1. Objetivo de negocio:

- Brindar a la empresa Deezer Europa Graph la oportunidad de robustecer su sistema de recomendación, basado en el género de sus usuarios y las interacciones entre ellos. Con el fin de proponer un perfilamiento sobre sus clientes, para futuras estrategias de mercadeo y reconocimiento en el mercado.

1.1.2. Criterio de éxito:

- Identificación de al menos 3 comunidades en las que se pueda diferenciar la participación de los usuarios de acuerdo con su género, para establecer plan de trabajo según la caracterización en común identificada.

1.2. Determinación de objetivos de minería de datos

1.2.1. Objetivo minería de datos

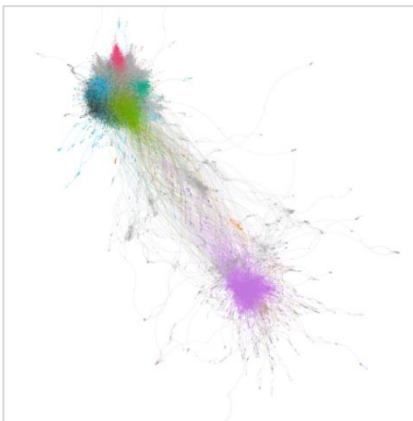
- Desarrollar un modelo basado en aprendizaje automático con grafos, que prediga el género de los usuarios utilizando las relaciones de seguimiento mutuo entre los usuarios y los artistas que le gustan a cada usuario.

1.2.2. Criterio de éxito

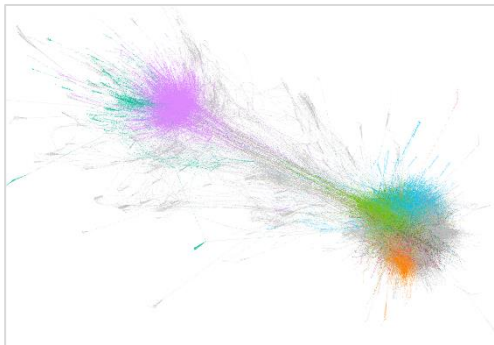
- Dado que es un caso de clasificación binaria, para evaluar la clasificación entre las dos clases (género M/F) se utilizará la evaluación del modelo(s) con su precisión (accuracy), con el objetivo de alcanzar o superar un 60%.

2. ENTENDIMIENTO DE LOS DATOS

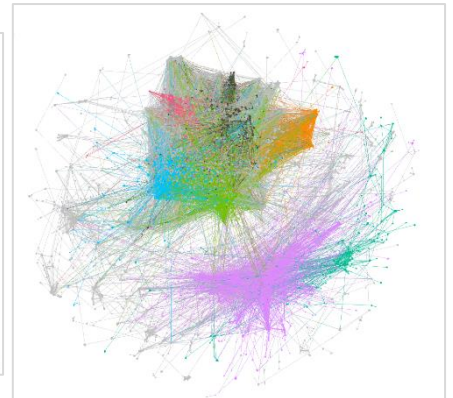
El Análisis de la Estructura del Grafo Deezer Europe, parte de los datos suministrados en el repositorio de Pytorch ([torch_geometric.datasets.DeezerEurope — pytorch_geometric documentation](https://pytorch-geometric.com/datasets/#DeezerEurope)), el cual consta de un total de 28.281 nodos representando a los usuarios europeos de la plataforma de música Deezer. Dentro de los atributos de estos, se identificaron 128 características por nodo, los cuales se infiere hacen referencia a características que comparten los usuarios a nivel de artistas o géneros musicales. Por otra parte, se identifican 185.504 aristas las cuales representan las relaciones de seguimiento mutuo entre los usuarios. A nivel de etiquetas, se identificaron 2 clases (1, 0), de acuerdo con la descripción de los datos, las etiquetas 0 y 1 probablemente representan los géneros masculino y femenino, respectivamente.



Distribución ForceAtlas2



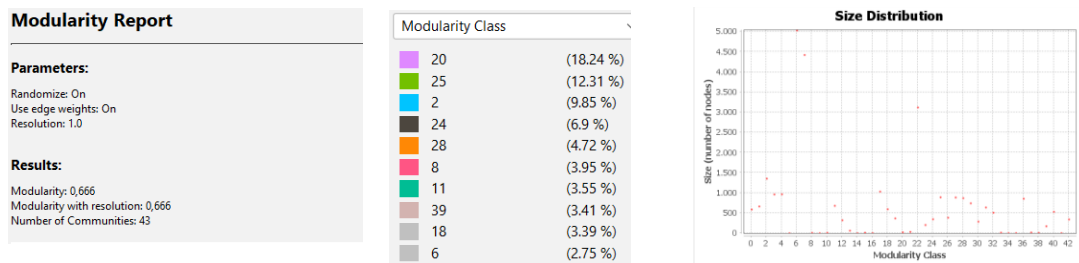
Distribución Yifan Hu's



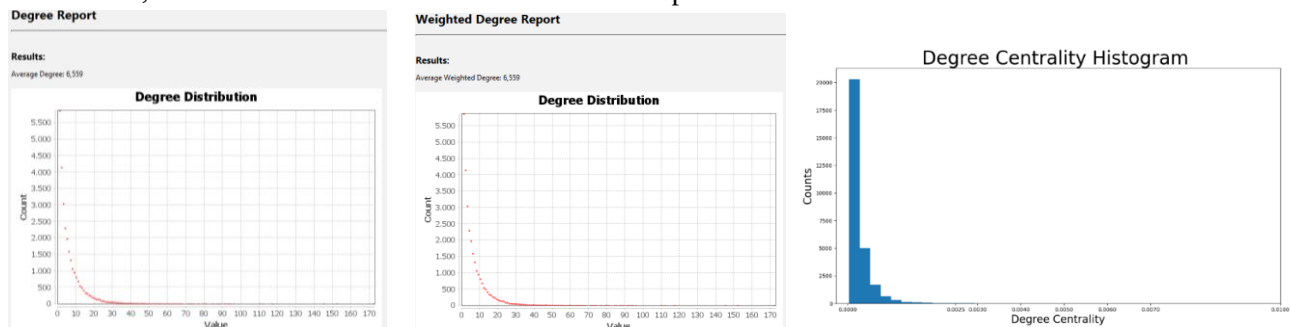
Distribución OpenOrd

Se graficó el grafo usando el aplicativo Gephi a través de diferentes distribuciones, lo que permitió una visualización clara de la estructura y la disposición de los nodos en la red. Cada una de estas distribuciones ayudó a resaltar las comunidades y las conexiones entre usuarios, logrando una representación más intuitiva de los grupos formados en la red.

Posteriormente, se realizó un análisis de modularidad para identificar las comunidades presentes en el grafo. Los resultados de este análisis indicaron un modularidad de 0.666, con un total de 43 comunidades detectadas. Este valor de modularidad refleja una estructura de red en la que los nodos tienden a agruparse en comunidades definidas, indicando afinidades o patrones de conexión compartidos entre los usuarios.



Se realizó un análisis de la visión general de la red, ejecutando las métricas de grado medio y grado medio ponderado, las cuales resultaron en un valor de 6.559. Esto indica que, en promedio, cada nodo (usuario) en el grafo tiene aproximadamente 6 o 7 conexiones. La coincidencia entre el grado medio y el grado medio ponderado sugiere que las conexiones entre los nodos tienen pesos uniformes o que los pesos de las aristas no están afectando significativamente la estructura general de la red, lo cual implica una distribución de conexiones bastante homogénea. Sin embargo, el histograma muestra que la mayoría de los nodos tienen una centralidad de grado baja, indicando que poseen pocas conexiones, mientras que unos pocos nodos tienen un valor de centralidad alto, actuando como nodos de ego. Esto sugiere una estructura de red desigual o de tipo escala libre, donde la conectividad se concentra en unos pocos nodos.



También, se ejecutó la métrica de diámetro de la red para analizar las distancias entre nodos, obteniendo los siguientes resultados:

DIAMETRO	RADIO	PROMEDIO DE CAMINOS
21	11	6.4498

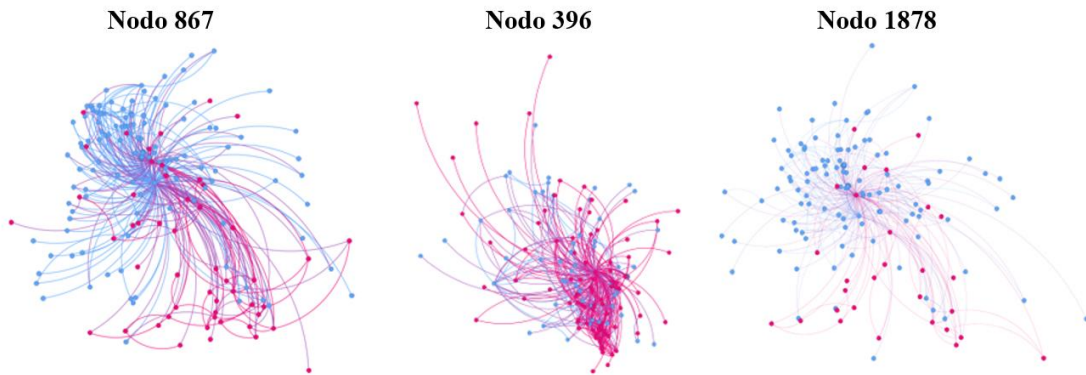
El diámetro representa la distancia más larga entre dos nodos cualquiera en la red, y un valor de 21 indica que, aunque la red tiene cierto nivel de cohesión, algunos nodos están relativamente alejados entre sí. El radio, que es la distancia mínima desde el nodo central a los nodos más lejanos, muestra que se puede alcanzar cualquier otro nodo desde el centro con un máximo de 11 pasos. La longitud promedio de camino de 6.4498 muestra que, en promedio, cualquier nodo está a unos 6 o 7 pasos de distancia de cualquier otro.

Se ejecutó la métrica de coeficiente medio de clustering, obteniendo:

AVERAGE CLUSTERING COEFFICIENT	TOTAL, DE TRIÁNGULOS EN LA RED
0.178	45034

Este resultado indica un nivel bajo de agrupamiento, lo que sugiere que los usuarios en la red no tienden a formar muchas conexiones directas entre sus conexiones. Aunque se identificaron una cantidad considerable de triángulos (conexiones entre tres nodos), el coeficiente de clustering bajo implica que, en relación con el tamaño de la red, el agrupamiento no es particularmente alto. Esto refleja una estructura de red en la que las conexiones entre los contactos de un usuario son relativamente débiles y dispersas.

Finalmente, se identificaron los tres principales nodos de ego, es decir a aquellos que presentan mayor cantidad de conexiones y centralidad. El nodo con el mayor grado es 867 con un grado de 172, y el promedio de grados por nodo es de 6.559. Se graficaron los nodos con un nivel de profundidad.



Obteniendo las siguientes proporciones, con respecto a los géneros (clases), de la siguiente manera:

- Nodo 867: Clase 1 (71%), clase 0 (29%)
- Nodo 396: Clase 0 (62%), clase 1 (38%)
- Nodo 1878: Clase 1 (79%), clase 0 (21%)

3. PREPARACIÓN DE LOS DATOS

Para ejecutar los modelos seleccionados, el proceso de preparación de datos incluyó una partición en tres conjuntos: entrenamiento (60%), prueba (20%) y validación (20%). Adicionalmente, para uno de los modelos desarrollados, se realizó la transformación de los inputs y labels del grafo, en un dataframe para su modelamiento.

4. MODELING & EVALUATION

MODEL	PARAMETERS	RESULT (ACCURACY)
Node2Vec	p=1; q=1 Embedding_dim=100, walk_length=10, context_size=10 walks_per_node=80 num_negative_samples=1	55.79%
NodeFeatures + Optuna	Perceptrón Multicapa (MLP) visto en clase con: Un cada oculta: linear Capa Salida: sigmoid, Optimizador Adam, Epochs:100 Hiperparámetros con Optuna: dim_h: 113, lr:5.7e-3, Weight Decay: 5e-3	65.10%
GNN + Adjacency Matrix	dim_in: 128, dim_h: 16, dim_out: 2 dropout_rate: 0.5, lr (learning rate): 0.01 optimizador Adam., weight_decay: 5e-5, epochs: 100	60.20%

Node2Vec + NodeFeatures	embedding_dim = 112, walk_length = 26 context_size = 12, walks_per_node = 7, p = 10 q = 1 Para el NodeFeatures, se mantuvo los parámetros del modelo 2.	50.8%
GCN	dim_in: 128, dim_h: 64, dim_out: 2, dropout_rate: 0.5, lr (learning rate): 0.001 optimizador Adam., weight_decay: 5e-5 epochs: 200, early_stopping: 10	62.88%
GAT	dim_h: 107, heads: 4, dropout: 0.2 lr: 0.002, weight_decay: 0.0002	63.12%
ACMII_GCN_PlusPlus	dim_in: 128 dim_h: 64 dim_out: 2 dropout_rate: 0.5 lr (learning rate): 0.01 optimizador: Adam weight_decay: 5e-4 epochs: 100	64.93%

Modelo 1: Node2Vec

La construcción del modelo utiliza Node2Vec para generar representaciones (embeddings) de los nodos y un clasificador Random Forest para la clasificación de nodos en el grafo. Node2Vec permite representar cada nodo en un espacio de características en función de su conectividad en el grafo. Inicialmente, se entrenó un clasificador Random Forest utilizando estos embeddings como entrada para predecir las etiquetas de los nodos, obteniendo una línea base de precisión. Posteriormente, se entrenó el modelo Node2Vec durante 100 épocas para optimizar los embeddings y mejorar el rendimiento del modelo en la tarea de clasificación.

Proceso de Entrenamiento

- Inicialización:

Node2Vec: Se configuró con los siguientes parámetros para generar embeddings de los nodos:

Hiperparámetro	Descripción	Valor Utilizado
embedding_dim	Dimensión del embedding	100
walk_length	Longitud de las caminatas	10
context_size	Tamaño del contexto	10
walks_per_node	Caminatas por cada nodo	80
p	Parámetro de sesgo (profundidad)	1
q	Parámetro de sesgo (amplitud)	1

Random Forest: Para el clasificador inicial, se utilizó el valor por defecto del clasificador RandomForestClassifier de Scikit-Learn, sin ajustes de hiperparámetros adicionales.

- Entrenamiento:

Node2Vec realiza caminatas aleatorias desde cada nodo para capturar su contexto, generando secuencias de nodos. Estas secuencias se utilizan para entrenar un modelo Skip-gram y obtener los embeddings de cada nodo.

Los embeddings generados inicialmente se usaron como entrada en un clasificador Random Forest, entrenado en un conjunto de entrenamiento (70%) y evaluado en los conjuntos de validación (15%) y prueba (15%) para obtener una línea base de precisión.

Después de esta línea base, se continuó entrenando Node2Vec durante 100 épocas adicionales para mejorar la calidad de los embeddings.

En cada época, el modelo Node2Vec optimiza los embeddings mediante una función de pérdida que compara nodos conectados y no conectados. Esta fase de entrenamiento ayuda a capturar mejor las relaciones complejas en el grafo.

- Resultados

Después de entrenar el modelo, se obtuvieron los siguientes resultados:

- Precisión en Validación: 53.7% (Random Forest)
- Precisión en Prueba: 55.79% (Después del entrenamiento de 100 épocas)

Estos resultados sugieren que el modelo Node2Vec, combinado con un clasificador Random Forest configurado con sus parámetros predeterminados, logró una precisión moderada en la clasificación de nodos en el grafo. La precisión obtenida es aceptable como punto de partida, aunque no es muy alta. Esto podría indicar que la estructura compleja del grafo no fue completamente capturada por los embeddings generados. Para mejorar el rendimiento, se propone explorar ajustes de hiperparámetros y arquitecturas de modelo más avanzadas, como redes neuronales convolucionales de grafos (GCNs), que podrían captar mejor la información estructural del grafo.

Modelo 2: Node Features + Optuna

Este modelo se desarrolló en dos partes principales: la definición de la estructura para una red neuronal de grafo en función de las características (features) de sus nodos y el mejoramiento de este a través del optimizador de Optuna, fases que se detallan a continuación:

- Establecimiento de la estructura para el modelo del grafo basados en las características de los nodos:

Se inició convirtiendo en un dataframe los datos de entrada y etiquetas del grafo escogido, con el fin de facilitar el análisis y manejo de estos de una manera tabular dentro del modelo. Posteriormente se genera una clase en PyTorch para generar una red neuronal sencilla, compuesta de una capa oculta lineal y una capa de salida sigmoideal, puesto que necesitamos clasificar entre dos clases. Luego de ello, se utilizó como función de pérdida, entropía cruzada binaria (BCELoss) y de optimizador Adam. Finalmente, se generaron dos métodos uno de validación y el otro de testeo para evaluar el rendimiento del modelo.

El modelo planteado previamente, se calificará según su precisión para ser comparable con los demás modelos y estar alineados a la métrica definida en el objetivo de minería de datos.

- Optimizador de Optuna

Posterior a la creación y estructuración del modelo, se hace uso de Optuna para obtener los hiperparámetros que mejor resultados den al modelo; con 50 intentos en el optimizador, y evaluando cada 20 épocas el rendimiento del accuracy; se obtuvo que el mejor resultado era con los siguientes hiperparámetros: **dim_h** = 113, es decir 113 neuronas en la capa oculta, con un **learning rate** de $5.7e^{-3}$ y un **weight_decay** de $5e^{-3}$. Con estos parámetros el modelo obtuvo una precisión de **65.10%**

Modelo 3: GNN + Adjacency Matrix

El modelo GNN comienza con una dimensión de entrada $\text{dim_in}=128$, lo que significa que cada nodo del grafo está representado por 128 características. La estructura del modelo se organiza en capas con la siguiente configuración:

- **Primera Capa SparseLayer (gnn1):** La primera capa es de tipo SparseLayer y toma la entrada $\text{dim_in}=128$ y la reduce a una representación de tamaño $\text{dim_h}=16$. La capa también recibe la matriz de adyacencia del grafo, permitiéndole modelar las relaciones entre los nodos. Tras esta capa, se aplica una normalización por lotes (bn1) para estabilizar el proceso de entrenamiento y luego una función de activación ReLU para introducir no linealidad.
- **Dropout:** Después de la activación ReLU, se aplica una capa Dropout con una probabilidad de 0.5, lo que significa que el 50% de las neuronas de esta capa se desactivan aleatoriamente durante el entrenamiento para ayudar a prevenir el sobreajuste.
- **Segunda Capa SparseLayer (gnn2):** La segunda capa de SparseLayer toma la salida de tamaño $\text{dim_h}=16$ de la primera capa y la reduce a $\text{dim_out}=2$, que es la dimensión de la salida final. Esta salida es luego normalizada con bn2 antes de pasar a la capa final.
- **Salida con Función de Activación Sigmoide:** La salida de la segunda capa pasa por una función sigmoid para convertir los valores en un rango de 0 a 1, generando probabilidades de pertenencia a cada una de las dos clases de salida.

Proceso de Entrenamiento

- **Inicialización:** Se utiliza el optimizador Adam con una tasa de aprendizaje $\text{lr}=0.01$ y un weight_decay de $5\text{e-}4$ para controlar la regularización de los pesos. La función de pérdida seleccionada es CrossEntropyLoss, ideal para problemas de clasificación con dos o más clases.
- **Entrenamiento:** En cada época el modelo realiza un pase hacia adelante (forward) para calcular la salida en base a los datos de entrada. Calcula la pérdida en función de los valores reales y predichos usando CrossEntropyLoss. Retropropaga los errores y actualiza sus parámetros con el optimizador Adam. Este ciclo se repite durante 100 épocas.
- **Evaluación en Validación:** Cada 20 épocas, se calcula la precisión y la pérdida en el conjunto de validación para monitorear el rendimiento del modelo y ajustar el entrenamiento en función de las métricas observadas.

Proceso de Prueba

La evaluación del modelo en el conjunto de prueba se lleva a cabo con la precisión (accuracy), arrojando un resultado de **60.20%**.

Optimización con Optuna

Optuna se utilizó para ajustar los hiperparámetros y optimizar el rendimiento del modelo. Durante el proceso de búsqueda, Optuna identificó varios conjuntos de parámetros y la mejor combinación de hiperparámetros fue en el Trial 12 con $\text{dim_h} = 48$, $\text{lr} = 0.0132$ y $\text{weight_decay} = 0.00038$, arrojando una accuracy de 62.99%; sin embargo, la mejora no fue significativa y el uso a nivel computacional es bastante.

Modelo 4 - Implementación de GCN con Pytorch Geometric

En la construcción inicial del modelo GCN, se definió una dimensión de entrada ($\text{dim_in}=128$). La primera capa convolucional (GCNConv) toma esta entrada y transforma la representación en un espacio reducido de

tamaño `dim_h=64`, aplicando una función de activación ReLU. Posteriormente, una segunda capa convolucional (GCNConv) reduce la representación de 64 dimensiones a la salida final con `dim_out=2`, correspondiente a las dos categorías de salida. Esta salida pasa por una función `log_softmax`. Al completar el proceso de entrenamiento y evaluación en el conjunto de prueba, el modelo inicial alcanzó una precisión (accuracy) de 62.57%.

Proceso de Entrenamiento

- **Inicialización:** El optimizador Adam se definió con una tasa de aprendizaje (`lr=0.001`) y un parámetro de regularización (`weight_decay=5e-5`). La función de pérdida empleada fue `CrossEntropyLoss`.
- **Entrenamiento:** En cada época, el modelo realiza un pase hacia adelante, calcula la pérdida, retropropaga los errores y actualiza sus parámetros. Este ciclo se repite a lo largo de 200 épocas.
- **Evaluación en Validación:** Cada 20 épocas, se evalúa el rendimiento del modelo en el conjunto de validación, midiendo la precisión y pérdida para ajustar y monitorear el proceso de entrenamiento.

Para mejorar el rendimiento, se implementó una segunda versión del modelo que incluía una capa de dropout con una tasa de 0.5 (`dropout_rate`). Después de aplicar la función de activación ReLU en la primera capa (`gcn1`), se agregó `h = self.dropout(h)`. Al ejecutar esta segunda versión del modelo con el nuevo diseño, la precisión (accuracy) mejoró ligeramente, alcanzando un 62.88%.

Para optimizar el modelo, se utilizó Optuna para realizar una búsqueda de los mejores hiperparámetros. Los parámetros que se optimizaron fueron los siguientes: `dim_h`, con un rango de 32 a 128; `dropout_rate`, con un rango de 0.2 a 0.6; `lr` (learning rate), con un rango de $1e-4$ a $1e-2$; y `weight_decay`, con un rango de $1e-5$ a $1e-3$. Optuna determinó los siguientes valores óptimos para estos hiperparámetros: `dim_h` de 115, `dropout_rate` de 0.3134459139864307, `lr` de 0.0027723656735508394 y `weight_decay` de 0.00023104203434717025. Sin embargo, al ejecutar el modelo utilizando estos hiperparámetros, se observó que la precisión (accuracy) final disminuyó ligeramente a 61.54%.

Modelo 5: Implementación de GAT con Pytorch Geometric

El código inicia con una configuración reproducible, estableciendo una semilla fija y dividiendo los datos en subconjuntos de entrenamiento (70%), validación (15%) y prueba (15%), usando máscaras para diferenciar los nodos en cada fase de entrenamiento y evaluación. La arquitectura del modelo GAT se define mediante una clase basada en `torch.nn.Module`, que incluye una capa de atención gráfica con múltiples cabezas en la primera capa para capturar relaciones complejas entre nodos, y una segunda capa de una sola cabeza de atención para reducir la representación a las clases objetivo. La activación ELU introduce no linealidad, mientras que el dropout evita el sobreajuste. En la salida, `log_softmax` genera una distribución log-probabilística, facilitando el cálculo de la pérdida de entropía cruzada. La función `fit()` gestiona el entrenamiento del modelo, optimizando con Adam y midiendo la pérdida y precisión en el conjunto de validación cada 20 épocas.

Para la optimización de hiperparámetros, Optuna explora un espacio de búsqueda que incluye la dimensión oculta (entre 16 y 128), el número de cabezas de atención (2 a 8), dropout (0.2 a 0.8), tasa de aprendizaje (10^{-4} a 10^{-2}), y weight decay (10^{-5} a 10^{-2}). Cada combinación de hiperparámetros se evalúa entrenando un modelo GAT en el conjunto de entrenamiento y validando su rendimiento en el conjunto de validación, buscando maximizar la precisión. La función principal (`main()`) ejecuta 50 pruebas con un límite de una hora, registrando la mejor combinación de hiperparámetros. Con estos parámetros óptimos, el modelo final se entrena y evalúa en el conjunto de prueba para obtener su precisión, reportando además el número de

pruebas, la mejor precisión de validación y los valores óptimos de cada hiperparámetro.

Parámetro	Valor
dim_h	107
heads	4
dropout	0,258603
lr	0,002336
weight_decay	0,000209

Mejores hiperparametros

Utilizando estos mejores hiperparámetros, se observó que el mejor valor de accuracy para el conjunto de validación fue de **63.43%**.

Modelo 6: ACMII_GCN_PlusPlus (Abstraído de la literatura)

La construcción del modelo utiliza ACMII_GCN_PlusPlus para generar representaciones de los nodos en el grafo y clasificar los nodos en dos categorías. ACMII_GCN_PlusPlus emplea capas convolucionales de grafos para capturar las relaciones entre los nodos en el grafo, y utiliza una capa de salida completamente conectada para la clasificación final. El modelo fue evaluado en términos de precisión en los conjuntos de entrenamiento, validación y prueba.

Proceso de Entrenamiento

1. Inicialización:

ACMII_GCN_PlusPlus: Se configuró con los siguientes parámetros para entrenar las representaciones de los nodos:

Hiperparámetro	Descripción	Valor Utilizado
hidden_channels	Dimensión de la capa oculta	64
dropout	Tasa de dropout	0.5
lr	Tasa de aprendizaje del optimizador	0.01
weight_decay	Decaimiento de peso	0.0005
loss_function	Función de pérdida	Cross Entropy

2. Entrenamiento:

El modelo realiza dos capas de convolución de grafos (GraphConv) para transformar las características de los nodos. Entre cada capa, se aplica una función de activación ReLU y un dropout con probabilidad $p=0.5$ para mitigar el sobreajuste.

La última capa fc convierte la salida de la segunda capa convolucional en una representación de dos clases.

Los nodos del grafo se dividen en tres conjuntos:

- Entrenamiento (70%): Para actualizar los pesos del modelo.
- Validación (15%): Para evaluar el rendimiento del modelo durante el entrenamiento.
- Prueba (15%): Para calcular la precisión final del modelo.

El entrenamiento se realiza durante 100 épocas, actualizando los pesos en cada época y calculando la precisión en los conjuntos de entrenamiento y validación. Se realiza un paso de retropropagación para minimizar la función de pérdida Cross Entropy. Esta función compara las predicciones del modelo (logits) con las etiquetas verdaderas (data.y) en el conjunto de entrenamiento, penalizando más fuertemente las predicciones incorrectas para mejorar el ajuste del modelo.

3. Resultados

- Después de entrenar el modelo, se obtuvieron los siguientes resultados:
- Precisión en Validación: 64.83%
- Precisión en Prueba: 64.93%
- Precisión en Entrenamiento: 80.43% (al final del entrenamiento)

Estos resultados muestran que el modelo ACMII_GCN_PlusPlus logró una precisión moderada en la clasificación de nodos en el grafo. Sin embargo, la diferencia notable entre la precisión en el conjunto de entrenamiento (80.43%) y en el conjunto de validación y prueba (alrededor de 65%) sugiere que el modelo está experimentando sobreajuste. Esto implica que el modelo está capturando patrones específicos del conjunto de entrenamiento que no se generalizan bien a los nodos no vistos en los conjuntos de validación y prueba.

Para abordar el problema de sobreajuste, podrían probarse técnicas adicionales de regularización, como incrementar la tasa de dropout, reducir la dimensión de la capa oculta.

5. DEPLOYMENT

Siguientes Pasos

1. En el desarrollo de los modelos presentados, hubo un reto en aumentar considerablemente el valor del **accuracy** de los modelos, visto que su máximo a nivel de benchmarking para este data set ha sido de 67.5% ([Deezer-Europe Benchmark \(Node Classification on Non-Homophilic \(Heterophilic\) Graphs\) | Papers With Code](#)). Después de revisar las métricas del grafo y posterior los resultados del modelamiento, se observó que una de las razones por las cuales el **accuracy** no podría llegar tan alto, puede ser por el bajo grado de centralidad en el grafo escogido, por lo cual, se dificulta el paso de información a través de este. Basado en ello, se propone:
 - a. **Vista minería de datos:** refinamiento de las características de los nodos para que estas estén mejor definidas y reflejen el comportamiento del usuario, con el fin de entender mejor las comunidades reflejadas y poder realizar un mejor manejo de datos. Así mismo, la definición o caracterización de las conexiones entre nodos, para mejorar el análisis entre las conexiones y su interpretación.
 - b. **Vista negocio:** segmentar a los usuarios para poder analizar de manera segregada los gustos y preferencias de los usuarios y poder llegar a estrategias más precisas de marketing o monetización para la compañía.
2. Generar un entorno para desplegar el modelo, con el fin de realizar recomendaciones en tiempo real o de manera periódica sobre las predicciones de género según la música. Esto podrá tener un impacto a nivel de negocio para las campañas publicitarias y, también como feedback y refuerzo para el modelo, con el fin de mantener un constante re-entrenamiento.

Estrategias

1. **Estrategias de Personalización y Marketing:** A partir del modelo desarrollado, Deezer puede implementar recomendaciones personalizadas basadas en el género musical y las interacciones entre usuarios, generando sugerencias precisas a nivel de playlists, artistas y géneros relacionados; permitiendo adaptar la experiencia de cada usuario y mejorar la satisfacción en la plataforma, aumentando el tiempo de uso y la lealtad hacia Deezer.
2. **Feedback y Mejora Continua:** Una vez puesto el modelo en producción, es esencial recopilar feedback de los usuarios sobre las recomendaciones entregadas por parte de la plataforma, en busca de detectar posibles problemas en la precisión de las recomendaciones y ajustar el modelo.
3. **Alianzas con Proveedores de Contenido Exclusivo:** Colaborar con artistas, discográficas y creadores de contenido para ofrecer música y material exclusivo a los usuarios de Deezer, como lanzamientos anticipados, sesiones en vivo y entrevistas exclusivas. Esto mejora la propuesta de valor y atrae a fanáticos que buscan experiencias únicas.

6. REPORTE TRABAJO EN EQUIPO

Actividad	David Hernando Ávila Jiménez	Erica Marcela Martínez Silva	Lina Marcela Rivas	Raúl Eduardo Vásquez Duarte	Sebastián Amaya Porras
Cargue e Interpretación de Base				X	
Graficar de Gephi y generar métricas		X			X
Entendimiento de Datos	X	X	X	X	X
Node2Vec				X	
NodeFeatures + Optuna			X		
GNN + Adjacency Matrix		X			
Node2Vec + NodeFeatures			X		
GCN					X
GAT	X				
ACMII_GCN_PlusPlus	X	X	X	X	X
Documentar	X	X	X	X	X
Presentación	X	X	X	X	X

Link del Modelo en Git:

https://github.com/eduartev93/Topicos_Avanzados_Analitica_LESDR/blob/Dev/P3%20-%20Graph%20Machine%20Learning/Proyecto_grafos_20241108_vf_2.ipynb